

# OS作业2-郭威威-2414308

## 实验目的

- 加深对操作系统系统调用的理解（如目录操作、文件操作）；
- 掌握 C 语言实现目录递归复制的方法；
- 对比不同语言（C/Python）的程序性能差异及原因。

## 实验环境

- 操作系统：Ubuntu 20.04 LTS；
- 工具版本：GCC 9.4.0、Python 3.8.10、Vim 8.1；
- 测试目录：Linux 内核 `linux-6.17.1`

## 实验代码

代码块

```
1  #include <stdio.h>    // 包含fopen、fclose、fread、fwrite、perror、printf (◆1-121)
2  #include <string.h>    // 包含snprintf、strcmp (◆1-122)
3  #include <sys/stat.h> // 包含stat、mkdir、S_ISDIR、S_ISREG (◆1-123)
4  #include <sys/types.h>
5  #include <dirent.h>    // 包含opendir、readdir、closedir (◆1-123)
6  #include <unistd.h>    // Unix标准函数 (◆1-123)
7  #include <errno.h>     // 错误码 (◆1-123)
8
9  // 函数声明：复制单个普通文件（源路径→目标路径）
10 int copy_file(const char *src_path, const char *dest_path) {
11     FILE *src_fp = NULL, *dest_fp = NULL;
12     char buf[4096]; // 缓冲区（4KB，平衡效率与内存占用）
13     size_t read_len;
14
15     // 1. 以二进制读模式打开源文件（rb：防止文本模式换行符转换问题，◆1-37）
16     src_fp = fopen(src_path, "rb");
17     if (src_fp == NULL) {
18         perror("fopen src_file failed"); // 打印错误信息 (◆1-96)
19         return -1;
20     }
21
22     // 2. 以二进制写模式创建目标文件（wb：覆盖已有文件，◆1-37）
```

```

23     dest_fp = fopen(dest_path, "wb");
24     if (dest_fp == NULL) {
25         perror("fopen dest_file failed");
26         fclose(src_fp); // 关闭已打开的源文件，避免资源泄漏 (◆1-39)
27         return -1;
28     }
29
30     // 3. 循环读取源文件内容并写入目标文件 (◆1-43、◆1-51)
31     while ((read_len = fread(buf, 1, sizeof(buf), src_fp)) > 0) {
32         if (fwrite(buf, 1, read_len, dest_fp) != read_len) {
33             perror("fwrite failed");
34             fclose(src_fp);
35             fclose(dest_fp);
36             return -1;
37         }
38     }
39
40     // 4. 检查fread是否因错误退出 (而非到达文件末尾)
41     if (ferror(src_fp)) {
42         perror("fread failed");
43         fclose(src_fp);
44         fclose(dest_fp);
45         return -1;
46     }
47
48     // 5. 关闭文件 (◆1-39)
49     fclose(src_fp);
50     fclose(dest_fp);
51     return 0;
52 }
53
54 // 函数声明：递归复制目录 (源目录→目标目录)
55 int copy_dir(const char *src_dir, const char *dest_dir) {
56     DIR *src_dirp = NULL;
57     struct dirent *dir_entry = NULL;
58     struct stat src_stat;
59     char src_path[1024], dest_path[1024]; // 存储拼接后的路径 (需足够长)
60
61     // 1. 打开源目录 (◆1-59)
62     src_dirp = opendir(src_dir);
63     if (src_dirp == NULL) {
64         perror("opendir src_dir failed");
65         return -1;
66     }
67
68     // 2. 获取源目录属性 (用于复制权限, ◆1-74)
69     if (stat(src_dir, &src_stat) == -1) {

```

```

70     perror("stat src_dir failed");
71     closedir(src_dirp); // 关闭目录，避免资源泄漏 (◆1-69)
72     return -1;
73 }
74
75 // 3. 创建目标目录 (权限与源目录一致，0755为示例，实际用src_stat.st_mode, ◆1-
80)
76 if (mkdir(dest_dir, src_stat.st_mode) == -1) {
77     // 忽略"目录已存在"错误 (EEXIST)，避免重复创建失败
78     if (errno != EEXIST) {
79         perror("mkdir dest_dir failed");
80         closedir(src_dirp);
81         return -1;
82     }
83 }
84
85 // 4. 遍历源目录中的所有条目 (◆1-64)
86 while ((dir_entry = readdir(src_dirp)) != NULL) {
87     // 跳过当前目录 (.) 和上级目录 (..)，避免无限递归
88     if (strcmp(dir_entry->d_name, ".") == 0 || strcmp(dir_entry->d_name,
89     "..") == 0) {
90         continue;
91     }
92
93     // 5. 拼接源路径和目标路径 (安全格式化，防止溢出, ◆1-83)
94     snprintf(src_path, sizeof(src_path), "%s/%s", src_dir, dir_entry-
95     >d_name);
96     snprintf(dest_path, sizeof(dest_path), "%s/%s", dest_dir, dir_entry-
97     >d_name);
98
99     // 6. 获取当前条目的属性 (判断是文件还是目录, ◆1-74)
100    if (stat(src_path, &src_stat) == -1) {
101        perror("stat entry failed");
102        continue; // 跳过无法获取属性的条目，继续处理其他条目
103    }
104
105    // 7. 判断条目类型：目录则递归复制，普通文件则直接复制 (◆1-29)
106    if (S_ISDIR(src_stat.st_mode)) { // 是目录
107        if (copy_dir(src_path, dest_path) == -1) {
108            printf("Warning: copy dir %s failed\n", src_path);
109        }
110    } else if (S_ISREG(src_stat.st_mode)) { // 是普通文件
111        if (copy_file(src_path, dest_path) == -1) {
112            printf("Warning: copy file %s failed\n", src_path);
113        }
114    }
115
116    // 文档要求不处理符号链接、设备文件等，故不添加其他类型判断

```

```

113     }
114
115     // 8. 关闭源目录 (◆1-69)
116     closedir(src_dirp);
117     return 0;
118 }
119
120 // 主函数：接收命令行参数 (源目录、目标目录)
121 int main(int argc, char *argv[]) {
122     // 检查命令行参数：需传入"源目录"和"目标目录" (目标目录格式：linux-6.17.1-[学号],
123     // ◆1-8)
124     if (argc != 3) {
125         fprintf(stderr, "Usage: %s <src_dir> <dest_dir>\n", argv[0]);
126         fprintf(stderr, "Example: %s linux-6.17.1 linux-6.17.1-2023001\n",
127             argv[0]);
128         return -1;
129     }
130
131     // 调用递归复制函数
132     if (copy_dir(argv[1], argv[2]) == 0) {
133         printf("Copy completed successfully!\n");
134         return 0;
135     } else {
136         printf("Copy failed!\n");
137         return -1;
138     }
139 }

```

## 实验截图

```

gww@gww-VMware-Virtual-Platform:~/os$ cd 第二次
gww@gww-VMware-Virtual-Platform:~/os/第二次$ touch mycopy.c
gww@gww-VMware-Virtual-Platform:~/os/第二次$ nano mycopy.c
gww@gww-VMware-Virtual-Platform:~/os/第二次$ gcc mycopy.c -o mycopy

```

```
gww@gww-VMware-Virtual-Platform:~/os/第二次$ time ./mycopy linux-6.17.1 linux-6.17.1-2414308
Copy completed successfully!

real    2m1.483s
user    0m2.721s
sys     0m40.786s
```

文件

最近

收藏

主文件夹

视频

图片

文档

下载

音乐

回收站

CDROM

Floppy Disk

Ubuntu 24...

其它位置

主文件夹 / os / 第二次

linux-6.17.1

linux-6.17.1-2414308

linux-6.17.1.tar.xz

mycopy

mycopy.c

已选中“linux-6.17.1.tar.xz” (153.4 MB)